

Advanced Lesson 8 - Exercises: Pattern Matching and Conditional Definitions

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: Why pattern matching belongs in functional arguments

Part 1. A definition written without a pattern, $f[x] = x^2$, attaches to the single symbol x and to nothing else. It is not a function; it is a stored value for one particular expression. Writing $f[x_] = x^2$ attaches to the pattern "any one argument", which is what a function of one variable actually means. Pattern matching is how a definition says which calls it applies to, and without it a definition applies to exactly one call.

```
Clear[g, x, y];  
g[x] = Sin[x];  
{g[x], g[y], g[0], g[ $\frac{\pi}{2}$ ]}
```

Part 2, an example that fails without pattern matching. Only $g[x]$ returns $\text{Sin}[x]$. $g[y]$, $g[0]$ and $g[\text{Pi}/2]$ all return unevaluated, because none of them is the symbol x . The definition is useless for every argument anybody would actually want to pass.

```
Clear[g, x, y];  
g[x_] = Sin[x];  
{g[x], g[y], g[0], g[ $\frac{\pi}{2}$ ]}
```

With the underscore all four evaluate: $\text{Sin}[x]$, $\text{Sin}[y]$, 0 and 1.

2: Why delayed assignment belongs in functional definitions

Part 1. Immediate assignment evaluates the right-hand side once, at definition time, when the pattern variable is still a bare symbol. Any right-hand side whose meaning depends on the argument having a value - a derivative, an integral, a random draw, a conditional test, a loop - is therefore computed against the wrong thing and frozen. Delayed assignment stores the right-hand side unevaluated and re-evaluates it per call, which is what a function body is supposed to do.

```
Clear[h1, h2, x];
h1[x_] = Length[x];
h2[x_] := Length[x];
{h1[{1, 2, 3}], h2[{1, 2, 3}]}
```

Part 2, an example that fails without delayed assignment. At definition time x is a symbol, and the length of a symbol is 0, so $h1$ stores the constant 0 and returns 0 for every argument. $h2$ stores $\text{Length}[x]$ unevaluated and returns 3, which is the answer.

3: Squares of integers, cubes of rationals, everything else unchanged

```
Clear[f, x, values];
f[x_Integer] := x^2;
f[x_Rational] := x^3;
f[x_] := x
??f
```

```
{f[ $\frac{3}{5}$ ], f[3], f[ $\frac{4}{2}$ ], f[3.1`]}
```

$f[3/5]$ is $27/125$, cubed as a rational. $f[3]$ is 9, squared as an integer. $f[4/2]$ is 4, not 8: Mathematica evaluates $4/2$ to the integer 2 before f ever sees it, so the integer rule applies. This is worth noticing - the pattern tests the head of the argument after evaluation, not the way it was typed. $f[3.1]$ is 3.1 unchanged, because a machine real is neither Integer nor Rational and falls through to the general rule.

```
values = Table[{x, f[x]}, {x, -5, 5,  $\frac{1}{2}$ }]
```

```
ListPlot[values]
```

The table steps in exact halves, so every other entry is a Rational and gets cubed while the entries between are Integers and get squared. The ListPlot shows the two interleaved families plainly: a parabola through the integer points and a cubic through the half-integer ones, crossing near the origin.

4: NumberQ against NumericQ

```
Clear[vals];
```

```
vals = {3,  $\frac{3}{4}$ , 3.1, 3 + 4 I,  $\sqrt{2}$ ,  $\pi$ , e};
```

```
NumberQ /@ vals
```

```
NumericQ /@ vals
```

The difference. NumberQ asks whether the expression already is an explicit number - Integer, Rational, Real or Complex. NumericQ asks whether it could be evaluated to a number by applying N.

NumberQ gives True for 3, 3/4, 3.1 and 3 + 4 I, and False for Sqrt[2], Pi and E: those three are exact symbolic expressions, not stored numbers.

NumericQ gives True for all seven.

For universal trigonometric evaluation NumericQ is the right choice. A test built on NumberQ would refuse Sin[Pi], Sin[Sqrt[2]] and Sin[E] - exactly the exact-symbolic arguments one most wants to handle - because it would decide they are not numbers. NumberQ answers a question about storage; NumericQ answers a question about meaning, and meaning is what a mathematical function should dispatch on.

5: VectorQ against ListQ against MatrixQ

```
Clear[objs];
```

```
objs = {3, {1, 2, 3}, {{1, 2}, {3, 4}},  
  {{{1}, {2}}, {{3, 4}}}, {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}}};
```

```
ListQ /@ objs
```

```
VectorQ /@ objs
```

```
MatrixQ /@ objs
```

The difference. ListQ asks only whether the head is List - it says nothing about shape or depth. VectorQ asks for a list none of whose elements is itself a list: a flat list, depth exactly one. MatrixQ asks for a list of equal-length lists of non-lists: a rectangular array, depth exactly two.

So: 3 fails all three. {1, 2, 3} passes all three except MatrixQ. {{1, 2}, {3, 4}} is a List and a Matrix but not a Vector, since its elements are lists. The ragged {{{1}, {2}}, {{3, 4}}} is only a List - its two elements have different lengths, so it is not rectangular, and it is three deep anyway. The 2 x 2 x 2 array is only a List: rank three is too deep for MatrixQ, which insists on exactly two.

6: A sign-switched sine, via Boolean tests

```
Clear[f, x];
f[x_] := Sin[x] /; Positive[x];
f[x_] := -Sin[x] /; ! Positive[x]
Plot[f[x], {x, -π, π}]
```

The condition operator "/" attaches a test to a definition: the rule only fires when the test returns True. Writing the second case as the negation of the first, rather than as Negative[x], is deliberate - it makes the two conditions exhaustive, so $x = 0$ is covered and no argument falls through unevaluated.

The plot is the absolute-value-like reflection: the negative half of the sine curve is flipped up, so the graph is a positive hump on each side of the origin.

7: Exercise 3 again, using conditional evaluation

```
Clear[f, x];
f[x_] := x^2 /; IntegerQ[x];
f[x_] := x^3 /; Head[x] === Rational;
f[x_] := x
{f[3/5], f[3], f[4/2], f[3.1`]}
ListPlot[Table[{x, f[x]}, {x, -5, 5, 1/2}]]
```

Same results as Exercise 3, reached differently. Instead of naming a head inside the pattern, each rule matches anything and then guards itself with a test. The two styles are interchangeable here; the pattern-head version is faster, because Mathematica can reject non-matching calls without evaluating a condition, but the conditional version generalises to tests that no head can express.

8: Exercise 6 again, using conditional evaluation

Exercise 6 was already written with `/;`, so there is nothing to change. It is repeated here in its Boolean-test form for the record; the two exercises coincide.

```
Clear[f, x];
f[x_] := Sin[x] /; x > 0;
f[x_] := -Sin[x] /; x ≤ 0
Plot[f[x], {x, -π, π}]
```

9: Exercise 3 again, using Which

```
Clear[f, x];
f[x_] := Which[IntegerQ[x], x2, Head[x] === Rational, x3, True, x]
{f[ $\frac{3}{5}$ ], f[3], f[ $\frac{4}{2}$ ], f[3.1`]}
ListPlot[Table[{x, f[x]}, {x, -5, 5,  $\frac{1}{2}$ }]]
```

Which takes alternating test-and-result pairs, evaluates the tests in order, and returns the result belonging to the first test that is True. The final True acts as the catch-all. One definition now carries all three cases, which reads better than three definitions when the cases are genuinely alternatives - and worse when they are separate ideas that might later be extended independently.

10: Exercise 6 again, using Which

```
Clear[f, x];
f[x_] := Which[x > 0, Sin[x], True, -Sin[x]]
Plot[f[x], {x, -π, π}]
```

11: Exercise 3 again, as a conditional replacement rule

```
Clear[x, rule3];
rule3 = {n_Integer → n2, n_Rational → n3, n_ → n};
{ $\frac{3}{5}$ , 3,  $\frac{4}{2}$ , 3.1`} /. rule3
```

```
Replace[{ $\frac{3}{5}$ , 3,  $\frac{4}{2}$ , 3.1`}, rule3, {1}]
```

```
ListPlot[Table[{x, Replace[x, rule3]}, {x, -5, 5,  $\frac{1}{2}$ }] ]
```

The same dispatch expressed as rules rather than as definitions. Delayed rules (":>") are required, not optional: with immediate rules the right-hand sides n^2 and n^3 would be evaluated once, at the moment the rule list was built, when n is still a bare symbol - the same trap as immediate assignment in Exercise 2. The rules are tried in order at each position, so the general catch-all must come last.

Now compare the two applications, because they do not agree. ReplaceAll returns the list unchanged. It works from the top down, and at the very top it finds the whole list $\{3/5, 3, 4/2, 3.1\}$, which matches the catch-all $n_$ and is replaced by itself. ReplaceAll never revisits what it has just replaced, so it never descends to the elements. The catch-all swallowed the container.

Replace with an explicit level specification of $\{1\}$ applies the rules to the elements and not to the list, and returns $\{27/125, 9, 4, 3.1\}$ as intended.

The ListPlot uses Replace on each scalar for the same reason, and reproduces the two interleaved curves from Exercise 3.

The general lesson is worth more than the exercise: a rule set containing an unguarded catch-all cannot be applied with ReplaceAll to a structure, only to an atom. Either restrict the level, or make the catch-all specific enough that the container does not match it.

12: Exercise 6 again, as a conditional replacement rule

```
Clear[x, rule6];
rule6 = {n_ >: Sin[n] /; n > 0, n_ >: -Sin[n]};
{-1, -0.5`, 0, 0.5`, 1} /. rule6
Replace[{-1, -0.5`, 0, 0.5`, 1}, rule6, {1}]
Plot[Replace[x, rule6], {x, - $\pi$ ,  $\pi$ }]
```

A condition can be attached to a replacement rule exactly as it can to a definition, and it is tested after the pattern matches but before the replacement is made. If the condition fails, the rule is abandoned at that position and the next rule is tried - which is why the unguarded fallback sitting second catches everything the first rule declines.

The two applications disagree again, and for the same reason as Exercise 11, though here the wrong answer is disguised. ReplaceAll matches the whole list against the second rule - the first declines, since a list is not greater

than zero - and returns `-Sin[{-1, -0.5, 0, 0.5, 1}]`, which `listability` turns into a list of numbers. It looks like an answer. It is the wrong answer: the positive entries have been negated, because the condition was tested once against the container instead of five times against the elements. `Replace` at level `{1}` gives the intended result, in which every entry is the odd reflection of the sine. The plot uses the same form, and `Plot`'s holding of its first argument means the replacement happens afresh at each sample point, with a genuine number in hand for the condition to test.